

Práctica: Teachable Machine

Clasificador de imágenes sin código — Duración: 3 horas

Curso: Machine Learning sin Código: Análisis Predictivo — Módulo 3.2 · UNEWEB

Objetivo general

Que el estudiante entrene, evalúe y documente un clasificador de imágenes funcional usando Teachable Machine, aplicándolo a un caso realista de negocio (control de calidad, clasificación de productos o detección de defectos), sin escribir una sola línea de código.

Requisitos previos

- Navegador (Chrome recomendado) con acceso a cámara (opcional, para pruebas en vivo)
- Cuenta de Google (para guardar el proyecto en Teachable Machine, opcional)
- Cuenta de Kaggle para descargar el dataset
- Acceso a: teachablemachine.withgoogle.com

Datasets sugeridos de Kaggle

Elige uno según el caso de uso que quieras trabajar con tus estudiantes (o deja que cada equipo elija el suyo):

Dataset (Kaggle)	Caso de uso	Por qué funciona bien
Casting Product Image Data for Quality Inspection	Control de calidad visual (defectuoso vs. correcto)	Imágenes ya organizadas en 2 carpetas (def_front / ok_front), tamaño pequeño, ideal para 3 horas
Surface Crack Detection	Detección de defectos simples	Clasificación binaria (grieta / sin grieta), imágenes pequeñas y numerosas
Fruits and Vegetables Image Recognition Dataset	Clasificación de productos por categoría	Múltiples clases claras, simula control de calidad en punto de venta
Garbage Classification	Clasificación de productos/materiales por categoría	6 clases bien diferenciadas (cartón, vidrio, metal, papel, plástico, basura)

Recomendación: para un grupo que ve el tema por primera vez, el dataset de Casting Product (defectuoso vs. correcto) es el más directo porque solo tiene 2 clases y conecta perfectamente con el ejemplo de “control de calidad visual” de la teoría.

Distribución del tiempo (180 minutos)

Tiempo	Actividad
0:00–0:10	Contexto y objetivo de la práctica: repaso del flujo de Teachable Machine (subir ejemplos → entrenar → probar → exportar) y del caso de negocio a resolver.
0:10–0:30	Descarga y curaduría del dataset: seleccionar 25–40 imágenes por clase y organizarlas en carpetas locales con nombres claros.

0:30–0:45	Crear el proyecto en Teachable Machine (Image Project → Standard image model) y crear una clase por categoría.
0:45–1:05	Cargar los ejemplos curados en cada clase, verificando que estén balanceadas.
1:05–1:30	Entrenar el modelo. Revisar el panel Advanced (epochs, batch size, learning rate) y discutir overfitting/underfitting.
1:30–1:50	Probar el modelo con la webcam o imágenes nuevas y registrar aciertos/errores con su % de confianza.
1:50–2:20	Iterar: agregar ejemplos a la clase con más errores y reentrenar. Comparar el nivel de confianza antes/después.
2:20–2:30	Exportar el modelo (TensorFlow.js) y guardar el enlace del modelo alojado en la nube de Google.
2:30–2:50	Documentar resultados en un informe corto (capturas, tabla de pruebas, mejora aplicada, reflexión de negocio).
2:50–3:00	Puesta en común: 2–3 equipos comparten su caso y su nivel de precisión final.

Detalle por bloque

Bloque 1 — Preparación (30 min)

Recordar brevemente qué es Teachable Machine y el flujo: subir ejemplos → entrenar → probar → exportar. Presentar el caso de negocio que van a resolver (ej. “una fábrica necesita separar piezas defectuosas automáticamente”).

Descargar el dataset elegido desde Kaggle y seleccionar manualmente 25–40 imágenes por clase (no usar el dataset completo: ralentiza la carga en el navegador y no es necesario para el objetivo pedagógico). Organizar las imágenes en carpetas locales, una por clase, con nombres claros (ok, defectuoso).

Bloque 2 — Entrenamiento (60 min)

Entrar a Teachable Machine → “Get Started” → “Image Project” → “Standard image model”. Crear una clase por categoría y renombrarlas según el dataset.

Subir las imágenes curadas a cada clase (arrastrar y soltar carpetas). Verificar que cada clase tenga una cantidad razonablemente balanceada de ejemplos — enfatizar el concepto de clases desbalanceadas visto en la teoría.

Pulsar “Train Model”. Mientras entrena, abrir el panel “Advanced” y mostrar los parámetros (epochs, batch size, learning rate), conectándolo con la teoría de overfitting/underfitting del Módulo 3.1. Discutir en grupo: ¿qué pasaría si subimos muy pocas imágenes por clase?

Bloque 3 — Evaluación e iteración (60 min)

Usar el panel “Preview” con la webcam o subiendo imágenes nuevas (no usadas en el entrenamiento) para probar clasificaciones. Registrar aciertos y errores en una tabla simple (imagen, clase real, clase predicha, % de confianza).

Con base en los errores detectados, agregar 5–10 imágenes adicionales a la clase que falló más, y reentrenar. Comparar el nivel de confianza antes/después del reentrenamiento. Objetivo pedagógico: que vivan en carne propia el ciclo iterativo del aprendizaje supervisado.

Exportar el modelo como TensorFlow.js (mostrar que existe la opción, aunque no se implemente en código) y guardar/copiar el enlace del modelo entrenado.

Bloque 4 — Cierre (30 min)

Cada estudiante o equipo entrega un breve informe con: captura del modelo entrenado y sus clases, tabla de pruebas (aciertos/errores, % de confianza), una mejora aplicada tras la iteración y una reflexión sobre en qué proceso real de su empresa/industria podrían aplicar esto.

Puesta en común: 2–3 equipos comparten su caso y su nivel de precisión final. Cierre con la pregunta: “¿qué limitaciones tiene este enfoque sin código frente a un modelo entrenado con código?”

Entregable final

Un informe corto (puede ser una diapositiva o media página) con: dataset usado, clases, capturas del entrenamiento y pruebas, y la reflexión de aplicabilidad de negocio.

Rúbrica sugerida

Criterio	Peso
Modelo entrenado con al menos 2 clases balanceadas	25%
Evidencia de prueba con imágenes nuevas (no usadas en el entrenamiento)	25%
Evidencia de al menos una iteración de mejora	25%
Reflexión de aplicación de negocio coherente	25%